# NLP-CS6370-PRJ

## BUILDING AND ENHANCING A MODULAR INFORMATION RETRIEVAL SYSTEM FOR NATURAL LANGUAGE QUERIES

**Aniket Khan[1], Anirudh Kalyan[1], Ashmit Sinha[1], Sameer[1]**

[1]Indian Institute of Technology Madras

## ABSTRACT

*In this project, we explore the various ways to improve information retrieval, on top of the initial implementation done as part of the earlier assignments. We detail implementation details, as well as include performance metrics of each hypothesis, evaluated on the Cranfield dataset. As part of our efforts, we hypothesize new systems (i) Using LSA on the term document matrix (ii) Lower dimension representation of the term document matrix using Autoencoders (iii) Lesk[1] algorithm for word sense disambiguation (iv) Query Expansion (v) SOTA transformer-based approach to improve on the baseline implementation. In this work we implement the aforementioned strategies, and report our findings.*

**Keywords: Information Retrieval, Word Sense Disambiguation, Query Expansion, TF-IDF, LSA, Autoencoder, BERT, NLP**

## NOMENCLATURE

*Roman letters*

| | |
|---|---|
| $f_{t,d}$ | Frequency of term $t$ in document $d$ |
| $N$ | Total number of documents in the corpus |
| $n_t$ | Number of documents containing term $t$ |

*Abbreviations and acronyms*

| | |
|---|---|
| TF-IDF | Term Frequency–Inverse Document Frequency |
| TF | Term Frequency |
| IDF | Inverse Document Frequency |
| LSA | Latent Semantic Analysis |
| SVD | Singular Value Decomposition |
| WSD | Word Sense Disambiguation |
| MAP | Mean Average Precision |
| nDCG | Normalized Discounted Cumulative Gain |
| F1-Score | Harmonic mean of precision and recall |
| P | Precision |
| R | Recall |
| BERT | Bidirectional Encoder Representations from Transformers |
| SOTA | State-of-the-Art |

*Datasets and corpora*

| | |
|---|---|
| PTB | Penn Treebank |
| WordNet | Lexical database of semantic relationships |
| Cranfield | Information retrieval benchmark dataset |

*Mathematical operators and functions*

| | |
|---|---|
| $\cap$ | Set intersection operator |
| $\lvert \cdot \rvert$ | Cardinality of a set |
| AveP | Average Precision |

*Superscripts and subscripts*

| | |
|---|---|
| $t$ | Term index or time step |
| $d$ | Document index |
| $q$ | Query index |
| $p$ | Cutoff rank for evaluation metrics like nDCG |

Project Report for CS6370 (Natural Language Processing) submitted on May 12, 2025.

## 1. BASIC INFORMATION RETRIEVAL SYSTEM

An Information Retrieval (IR) system is designed to facilitate the retrieval of relevant documents from a large corpus based on user queries. The fundamental goal is to match the user's query with the most relevant documents, typically ranked by relevance. A basic IR system follows several steps:

- **Indexing:** The documents in the corpus are indexed, typically by extracting terms (words) and storing them in a data structure, such as an inverted index. The index maps terms to the documents in which they appear, facilitating efficient search operations.
- **Query Processing:** When a user submits a query, the system processes it to identify the terms that should be matched with the documents in the corpus. Query preprocessing may involve tasks such as stemming, stop word removal, and term weighting.
- **Retrieval:** The system retrieves documents that contain the terms specified in the query. The retrieved documents

are ranked based on a relevance measure, which typically considers the frequency of query terms in the document, their inverse document frequency (IDF), and possibly the query's length.

- **Ranking:** Retrieved documents are ranked based on their relevance to the query. The relevance is often determined using a similarity measure such as cosine similarity between the query and the document vectors.

The traditional approach to indexing and retrieving documents is based on the vector space model, where both documents and queries are represented as vectors in a high-dimensional space. The documents are ranked based on their similarity to the query vector. One common method to weight the terms in the vector is the Term Frequency-Inverse Document Frequency (TF-IDF) model.

The TF-IDF weighting scheme assigns a weight to each term based on how frequently it appears in a document (Term Frequency) and how common it is across the entire corpus (Inverse Document Frequency). While TF-IDF is widely used for its simplicity and effectiveness in many cases, it has limitations in capturing the semantic relationships between terms, as it treats terms as independent and ignores their contextual meaning.

## 2. INTRODUCTION

Our current Information Retrieval (IR) system employs Term Frequency-Inverse Document Frequency (TF-IDF) based indexing, as described by Qaiser and Ali (2018), which represents documents as bag-of-words vectors. In this representation, each word is considered an orthogonal basis vector in a high-dimensional vector space, thereby ignoring any semantic similarity between terms. While the model has delivered reasonably good performance in general, it fails to retrieve even a single relevant document within the top-10 results for certain queries. These limitations underscore the need for more sophisticated word and sentence representation techniques.

TF-IDF is a statistical measure used to evaluate the importance of a word in a document relative to a collection of documents (the corpus). It is calculated as the product of two terms: term frequency (TF) and inverse document frequency (IDF). Mathematically, TF-IDF for a term $t$ in a document $d$ is defined as:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t) \tag{1}$$

where

$$\text{TF}(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}, \quad \text{IDF}(t) = \log\left(\frac{N}{n_t}\right) \tag{2}$$

Despite its simplicity and interpretability, the TF-IDF model suffers from several critical drawbacks: (i) the high-dimensional and sparse nature of the vectors often results in low cosine similarity scores, especially when the vocabulary of the query and document do not directly overlap; (ii) the model has low recall, as it is incapable of recognizing synonyms or semantically related terms; and (iii) it exhibits low precision due to the bag-of-words representation, which ignores word order and contextual meaning.

In this project, we aim to address these limitations. We begin by identifying and analyzing the key issues inherent in TF-IDF-based indexing, followed by hypothesizing their causes and proposing potential solutions. All experiments and evaluations will be conducted using the Cranfield dataset as mentioned earlier.

To provide a concise overview of the rationale behind our hypotheses, we outline the chronological development of our experimental approaches. Our analysis began with the implementation of Latent Semantic Analysis (LSA), motivated by the need to reduce the high dimensionality introduced by TF-IDF vectorization. Recognizing the limitations of traditional linear dimensionality reduction techniques, we further hypothesized that an autoencoder-based approach could more effectively learn compact, non-linear representations of the high-dimensional vector space.

Another critical factor affecting retrieval performance was the semantic ambiguity of words. To address this, we hypothesized that incorporating Word Sense Disambiguation (WSD) techniques would enhance retrieval accuracy. We implemented both a basic and an improved version of the Lesk algorithm to evaluate this hypothesis.

In a similar vein, we posited that enriching the query through query expansion would provide additional contextual information, thereby improving the model's ability to retrieve relevant documents. Consequently, we integrated a query expansion step into the query ingestion pipeline.

Lastly, we hypothesized that leveraging state-of-the-art (SOTA) language models, which inherently capture rich contextual and semantic information, would yield significant improvements in performance compared to traditional methods.

The results of our experiments, along with an evaluation of each approach using defined performance metrics, are presented in the following sections.

## 3. PERFORMANCE METRICS

It is important to detail the precise performance metrics that we use to evaluate our hypothesis. We aim to create a holistic evaluation scheme that ensures the implementation is evaluated on all use cases. To ensure this, we implement the following 5 performance metrics:

### 3.1. Precision

Precision measures the accuracy of the retrieval process by calculating the proportion of retrieved documents that are actually relevant to the query. It reflects how many of the retrieved results are useful to the user. High precision indicates that the system retrieves mostly relevant documents and avoids presenting irrelevant ones.

$$\text{Precision} = \frac{|\text{Relevant} \cap \text{Retrieved}|}{|\text{Retrieved}|} \tag{3}$$

### 3.2. Recall

Recall evaluates the system's ability to retrieve all relevant documents from the corpus. It is defined as the ratio of relevant documents retrieved to the total number of relevant documents available. A high recall means that the system is comprehensive

in finding relevant results, though it may retrieve some irrelevant ones as well.

$$\text{Recall} = \frac{|\text{Relevant} \cap \text{Retrieved}|}{|\text{Relevant}|} \tag{4}$$

### 3.3. F1-Score

The F1-score provides a balanced measure of both precision and recall by computing their harmonic mean. It is particularly useful when there is a need to balance the trade-off between retrieving as many relevant documents as possible (recall) and minimizing the inclusion of irrelevant ones (precision). A high F1-score indicates strong overall performance.

$$\text{F1-Score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{5}$$

### 3.4. Mean Average Precision (MAP)

MAP is a commonly used metric for evaluating ranked retrieval results. It computes the mean of the average precision scores across all queries. Average Precision (AP) at a given rank $k$ takes into account the order in which relevant documents are retrieved. MAP rewards systems that not only retrieve relevant documents but also rank them higher in the result list.

$$MAP@k = \frac{\sum_{q=1}^{|Q|} AveP(q)}{|Q|} \tag{6}$$

### 3.5. Normalized Discounted Cumulative Gain (nDCG)

nDCG is a ranking-aware metric that evaluates the quality of the entire list of retrieved documents by assigning higher scores to relevant documents that appear earlier in the ranking. It uses a logarithmic discount to penalize relevant documents appearing lower in the ranking. nDCG is normalized by the ideal DCG (IDCG), which is the maximum possible DCG for a perfectly ordered result list. This makes nDCG a robust metric for comparing across different queries and systems.

$$nDCG_p = \frac{DCG_p}{IDCG_p} \tag{7}$$

## 4. DATASETS

To implement and evaluate our algorithms, we utilize three well-established datasets: the Cranfield dataset, WordNet, and the Penn Treebank dataset. These datasets serve different but complementary purposes in our experiments.

- **Cranfield Dataset:** The Cranfield collection is a standard dataset in the field of information retrieval. It consists of approximately 1,400 abstracts of aeronautical engineering documents, each accompanied by a query and relevance judgments. The dataset is particularly valuable for evaluating document retrieval systems and has been widely used for benchmarking search algorithms and ranking techniques.
- **WordNet:** WordNet is a large lexical database of English developed at Princeton University. Nouns, verbs, adjectives, and adverbs are grouped into sets of cognitive synonyms (synsets), each expressing a distinct concept. WordNet also captures various semantic relationships between these synsets, such as hypernyms, hyponyms, and meronyms, making it highly useful for natural language understanding, word sense disambiguation, and semantic similarity tasks.
- **Penn Treebank (PTB):** The Penn Treebank is a corpus of annotated English text that includes syntactic and part-of-speech information. It contains a large number of sentences from sources such as the Wall Street Journal, annotated with detailed syntactic structure in the form of phrase-structure trees. The PTB is widely used for training and evaluating models in syntactic parsing, part-of-speech tagging, and other NLP tasks that require grammatical information.

## 5. PERFORMANCE

To evaluate the performance of our baseline TF-IDF based Information Retrieval system, we conducted experiments on the Cranfield dataset using the five performance metrics described previously. The evaluation was carried out at cutoff points $k = 1$ to $k = 10$, and the average scores over all queries were computed.

### 5.1. Metric Trends over Top-K Retrieval

Figure 1 shows the variation of precision, recall, F1-score, MAP, and nDCG with increasing values of $k$. As expected, precision decreases with larger $k$, while recall improves due to the inclusion of more documents. F1-score initially rises but later stabilizes, reflecting the trade-off between precision and recall. MAP and nDCG demonstrate a consistent decline in ranking quality, though nDCG remains comparatively high, indicating the system retrieves some relevant documents near the top of the ranking.

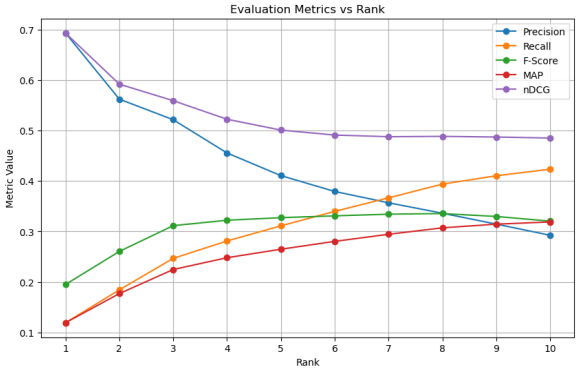



**FIGURE 1: Performance metrics of baseline TF-IDF model over different top-K values.**

### 5.2. Tabular Summary of Metrics

Table 1 provides the exact values of the performance metrics at each value of $k$ from 1 to 10. The system achieves high

precision at low $k$, but the recall and F1-score indicate room for improvement, motivating the need for more advanced semantic models.

| Top-K | Precision | Recall | F1-Score | MAP | nDCG |
|---|---|---|---|---|---|
| 1 | 0.693 | 0.119 | 0.195 | 0.119 | 0.693 |
| 2 | 0.562 | 0.184 | 0.260 | 0.177 | 0.592 |
| 3 | 0.521 | 0.247 | 0.312 | 0.225 | 0.559 |
| 4 | 0.456 | 0.281 | 0.322 | 0.248 | 0.522 |
| 5 | 0.411 | 0.311 | 0.327 | 0.265 | 0.501 |
| 6 | 0.379 | 0.340 | 0.331 | 0.280 | 0.491 |
| 7 | 0.357 | 0.367 | 0.334 | 0.294 | 0.488 |
| 8 | 0.336 | 0.394 | 0.335 | 0.307 | 0.489 |
| 9 | 0.315 | 0.410 | 0.330 | 0.314 | 0.487 |
| 10 | 0.292 | 0.423 | 0.320 | 0.319 | 0.485 |

**TABLE 1: Evaluation of baseline system metrics at different top-K cutoff values.**

## 6. IMPROVING THE INFORMATION RETRIEVAL SYSTEM

### 6.1. Introduction

The goal of our project is to design an improved information retrieval (IR) system capable of accurately ranking documents in response to natural language queries. Starting from a traditional TF-IDF-based Vector Space Model (VSM), we identified various failure cases—such as inability to handle synonyms, polysemous terms, spelling errors, and overly literal query matching—which motivated the need for enhanced semantic understanding and linguistic robustness. Our improvements draw from a combination of classical NLP, neural representations, and hybrid techniques.

### 6.2. Limitations of the Basic VSM (TF-IDF)

The TF-IDF approach, while efficient and interpretable, suffers from several well-known limitations:

- **Synonymy**: Query and documents may use different words for the same concept (e.g., *physician* vs. *doctor*), leading to missed matches.
- **Polysemy**: A query term like *bank* may refer to a financial institution or a river bank, leading to irrelevant document retrieval.
- **Spelling Errors**: Misspelled queries or document terms result in zero overlap.
- **Context Ignorance**: TF-IDF treats terms independently, ignoring word order and phrase-level meaning.
- **Shallow semantics**: It lacks the ability to generalize across related concepts.

To address these, we built and rigorously evaluated multiple techniques that each address a subset of these issues.

### 6.3. Proposed Enhancements

We implemented the following improvements, each addressing specific limitations of the baseline model:

**6.3.1. Latent Semantic Analysis (LSA)** Latent Semantic Analysis (LSA)[2] addresses challenges like synonymy and hidden topic structures in information retrieval by mapping terms and documents into a shared latent semantic space. This is achieved by applying truncated Singular Value Decomposition (SVD) to the term-document matrix, reducing its dimensionality and revealing underlying semantic structures.

We first examined how the number of latent dimensions affects the variance captured by the model. This is crucial to balance semantic expressiveness and computational efficiency. As shown in Figure 2, the variance captured increases rapidly in the first few components and then saturates, indicating that a relatively small number of dimensions (e.g., rank 400) can retain a substantial portion of the underlying structure.

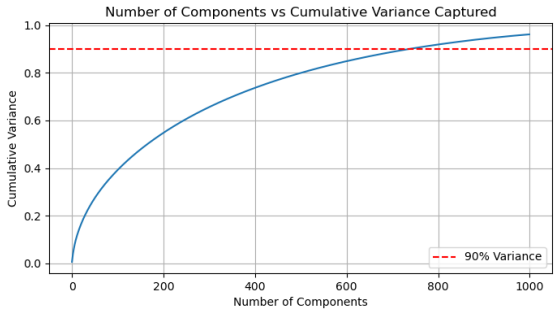



**FIGURE 2: Number of components vs. explained variance in the LSA model.**

To further understand component importance, we plotted the singular values across dimensions (Figure 3). The sharp decline in singular values confirms that most of the informative structure resides in the top few dimensions.

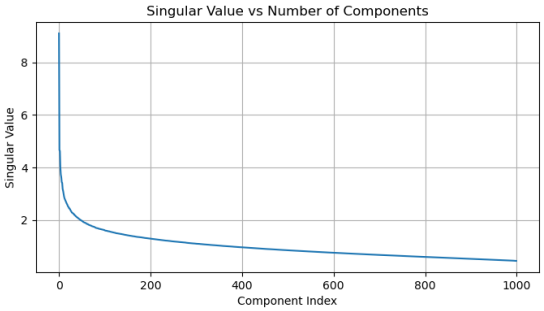



**FIGURE 3: Singular values across components in the LSA model.**

We evaluated retrieval performance using NDCG and MAP scores across varying ranks. Figure 4 shows that NDCG improves with increasing rank but stabilizes around rank 400. A similar trend is evident from the MAP scores in Figure 5, indicating diminishing returns beyond this dimensionality.

A summary of metrics across different hidden dimensions is provided in Table 1. The LSA model achieves optimal performance at rank 400, after which further increases in dimensionality yield marginal or no improvements. This suggests the

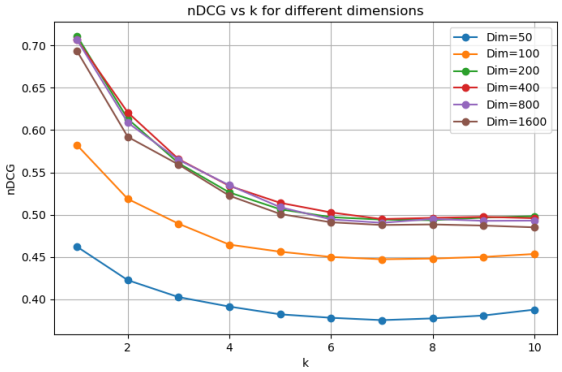



**FIGURE 4: NDCG score across different values of number of retrievals(k). Best performance was achieved at hidden dimension 400.**

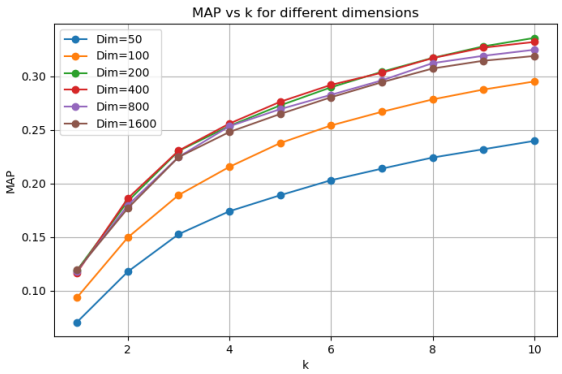



**FIGURE 5: MAP scores for the LSA model at different ranks.**

term-document matrix exhibits mostly linear structure, which the SVD-based method captures effectively.

| Hidden Dim | P@1 | MAP@10 | nDCG@10 | F1 (avg) | R@10 |
|---|---|---|---|---|---|
| 50 | 0.524 | 0.2474 | 0.5244 | 0.2742 | 0.3629 |
| 100 | 0.604 | 0.3033 | 0.4643 | 0.3248 | 0.4277 |
| 200 | 0.684 | 0.3308 | 0.5507 | 0.3359 | 0.4376 |
| **400** | **0.702** | **0.3296** | **0.5330** | **0.3377** | **0.4340** |
| 800 | 0.707 | 0.3247 | 0.4938 | 0.3264 | 0.4298 |
| 1600 | 0.693 | 0.3189 | 0.4851 | 0.3204 | 0.4235 |

**TABLE 2: Summary of retrieval metrics per hidden dimension: Precision@1, MAP@10, nDCG@10, average F1 (Ranks 1–10), and Recall@10 for SVD-based LSA**

**6.3.2. Autoencoder-based LSA** In contrast to the linear structure of LSA, an autoencoder[3]w-based approach captures non-linear relationships in the document space. Here, a document reconstruction autoencoder is trained, and its bottleneck layer is used as the latent representation for retrieval.

We studied how dimensionality affects this model's retrieval performance. As illustrated in Figure 6, the NDCG scores improve with increased dimensions, with the best performance observed at hidden dimension 800. This reflects the autoencoder's ability to capture richer semantic structures at higher dimensionalities.

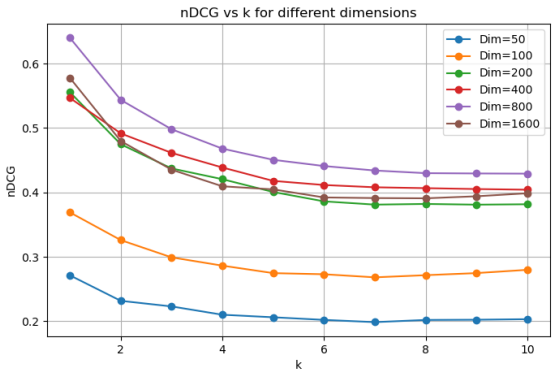



**FIGURE 6: NDCG score across different values of number of retrievals(k) in the Autoencoder model. Best performance was achieved at hidden dimension 800.**

The MAP scores (Figure 7) show a similar trend, further supporting the effectiveness of using deeper representations for capturing semantic similarity in retrieval.

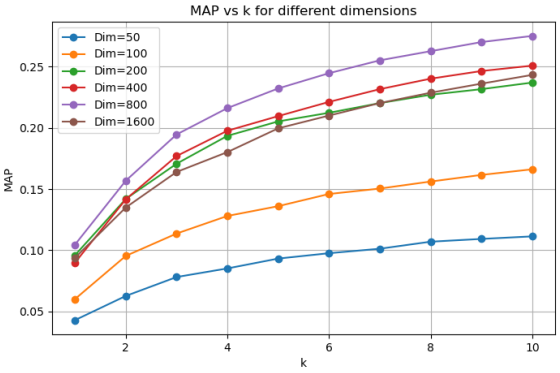



**FIGURE 7: MAP scores for the Autoencoder model at different ranks.**

Table 2 summarizes the performance of the autoencoder across hidden dimensions. While it achieves competitive performance—particularly at higher ranks—its improvement over LSA is marginal in this setup. This indicates that the dataset's structure may be mostly linear, limiting the gains from complex non-linear modeling.

Overall, while autoencoders offer a more flexible non-linear alternative to SVD, the LSA model demonstrates better performance in this context with fewer dimensions, making it a computationally attractive and semantically effective choice.

**6.3.3. Lesk Algorithm (Word Sense Disambiguation)** To address polysemy in queries, we implemented a classical Word Sense Disambiguation (WSD) technique based on the Lesk algorithm. This method selects the correct sense of a word by maximizing the overlap between the dictionary definition (gloss) of each candidate sense and the context in which the word appears.

| Hidden Dim | P@1 | MAP@10 | nDCG@10 | F1 (avg) | R@10 |
|---|---|---|---|---|---|
| 50 | 0.342 | 0.1421 | 0.3422 | 0.1654 | 0.2253 |
| 100 | 0.364 | 0.1784 | 0.3455 | 0.2126 | 0.2535 |
| 200 | 0.511 | 0.2290 | 0.4239 | 0.2510 | 0.3117 |
| 400 | 0.595 | 0.2639 | 0.4476 | 0.2768 | 0.3389 |
| **800** | **0.626** | **0.2677** | **0.4555** | **0.2895** | **0.3571** |
| 1600 | 0.622 | 0.2644 | 0.4408 | 0.2831 | 0.3527 |

**TABLE 3: Summary of retrieval metrics per hidden dimension: Precision@1, MAP@10, nDCG@10, average F1 (Ranks 1–10), and Recall@10 for auto-encoder based LSA**

Formally, for a word $w$ with senses $S = \{s_1, s_2, \ldots, s_n\}$ and context $C$, we compute:

$$\hat{s} = _{s_i \in S} \ |gloss(s_i) \cap C|$$

where $gloss(s_i)$ denotes the set of lemmatized, stopword-removed tokens from the dictionary definition of $s_i$. We use WordNet as the sense inventory. If no significant overlap (threshold $\theta$) is found, we default to the most frequent sense. The disambiguated query is then embedded using simple TF-IDF.

We evaluated the performance using standard retrieval metrics (Table 4) and visualized results in Figure 8.

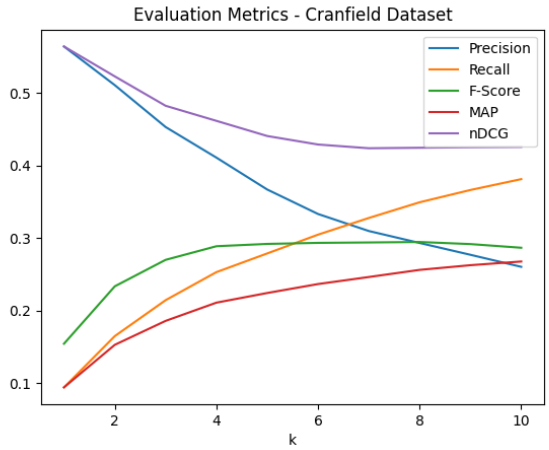



**FIGURE 8: Performance metrics of the WSD Lesk model over different top-K values.**

The Lesk model underperforms compared to embedding-based baselines. This likely stems from its reliance on exact lexical overlap, which leads to *data sparsity*—glosses are often too short to find meaningful overlaps with the query context. Furthermore, the model treats all matching words equally, ignoring the term frequency advantages exploited by TF-IDF. As a result, it fails to prioritize discriminative context terms. These limitations motivate our next hypothesis: integrating statistical weighting and embedding-based disambiguation methods for richer semantic matching.

**6.3.4. Improved Lesk (Extended Lesk with Embeddings)** To overcome the limitations of the basic Lesk algorithm, we introduce an enhanced version incorporating selective disambiguation, context enrichment, and semantic extensions. This

| Top-k | Precision | Recall | F1 | MAP | nDCG |
|---|---|---|---|---|---|
| 1 | 0.5644 | 0.0941 | 0.1543 | 0.0941 | 0.5644 |
| 2 | 0.5111 | 0.1648 | 0.2335 | 0.1528 | 0.5232 |
| 3 | 0.4533 | 0.2144 | 0.2701 | 0.1858 | 0.4826 |
| 4 | 0.4111 | 0.2531 | 0.2889 | 0.2110 | 0.4618 |
| 5 | 0.3671 | 0.2789 | 0.2921 | 0.2243 | 0.4409 |
| 6 | 0.3333 | 0.3048 | 0.2935 | 0.2367 | 0.4292 |
| 7 | 0.3098 | 0.3278 | 0.2941 | 0.2464 | 0.4240 |
| 8 | 0.2933 | 0.3495 | 0.2947 | 0.2562 | 0.4247 |
| 9 | 0.2770 | 0.3664 | 0.2918 | 0.2627 | 0.4254 |
| 10 | 0.2604 | 0.3814 | 0.2868 | 0.2678 | 0.4258 |

**TABLE 4: Evaluation of Lesk word sense disambiguation metrics at different cutoff levels (Top-K).**

model focuses on disambiguating only ambiguous and high-importance terms, thereby reducing noise and unnecessary computations. The context is extended to include surrounding sentences and domain-specific terms, enriching the disambiguation signals.

The key innovation lies in using *extended glosses* drawn from WordNet, which include not just the synset definition but also usage examples and semantically linked terms. We further employ a *dual-indexing strategy*, storing both original and sense-annotated document representations. These are combined using a *weighted ensemble ranking* mechanism. The model also applies *confidence thresholding* to revert to the baseline representation when disambiguation is uncertain, *sense frequency bias* to favor common senses, and *term importance weighting* to prioritize disambiguation of discriminative terms.

This multi-pronged strategy allows us to generate more semantically accurate document vectors for retrieval, though it does increase the computational cost.

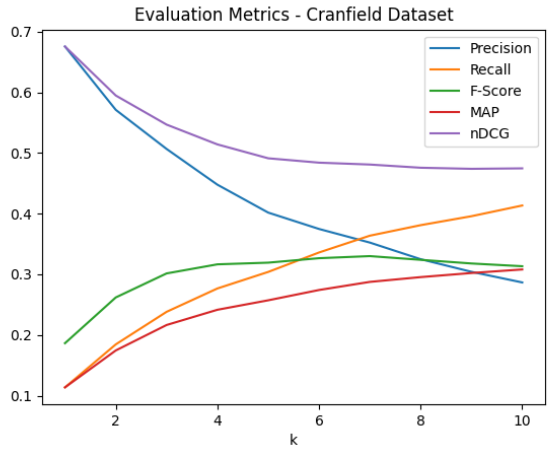



**FIGURE 9: Performance metrics of the improved WSD Lesk model over different top-K values.**

As shown in Figure 9, the improved Lesk model outperforms the classical Lesk across all metrics, with notable improvements in nDCG and MAP, especially in the top-K range. This reflects its improved ability to rank semantically relevant documents higher.

| Top-K | Precision | Recall | F1-score | MAP | nDCG |
|---|---|---|---|---|---|
| 1 | 0.6756 | 0.1137 | 0.1865 | 0.1137 | 0.6756 |
| 2 | 0.5711 | 0.1846 | 0.2620 | 0.1746 | 0.5947 |
| 3 | 0.5067 | 0.2383 | 0.3015 | 0.2165 | 0.5468 |
| 4 | 0.4478 | 0.2767 | 0.3165 | 0.2416 | 0.5142 |
| 5 | 0.4018 | 0.3040 | 0.3193 | 0.2572 | 0.4913 |
| 6 | 0.3748 | 0.3360 | 0.3266 | 0.2742 | 0.4841 |
| 7 | 0.3524 | 0.3637 | 0.3301 | 0.2876 | 0.4810 |
| 8 | 0.3250 | 0.3811 | 0.3240 | 0.2954 | 0.4757 |
| 9 | 0.3042 | 0.3958 | 0.3179 | 0.3022 | 0.4739 |
| 10 | 0.2867 | 0.4135 | 0.3134 | 0.3081 | 0.4747 |

**TABLE 5: Evaluation metrics at different top-K levels for the improved Lesk algorithm.**

While the computational time increases substantially (303.8s vs. 30.5s for basic Lesk), the gains in ranking quality justify the cost for applications requiring higher semantic fidelity.

**6.3.5. Query Expansion via Synonyms and Embedding Neighbors** Traditional TF-IDF-based retrieval methods often fail to return relevant documents when different but semantically similar terms are used in queries and documents. To bridge this vocabulary gap, we apply a query expansion technique that augments the original query with semantically related terms derived from external knowledge sources.

We expand the original query by adding a limited number of semantically related terms for each query word. These related terms are sourced from:

- **WordNet-based synonyms:** For each word in the query, we retrieve its synonyms from WordNet and include only the top-$k$ synonyms (e.g., $k = 2$ or 3) to avoid introducing noise.
- **Embedding-based neighbors (optional):** To capture distributional semantics, we also identify nearby terms in the embedding space using models such as Word2Vec[4] or GloVe, and include the most relevant ones.

To prevent topic drift (where unrelated terms are introduced due to polysemy or weak semantic links), we apply filtering based on:

- **Part-of-speech compatibility:** Ensuring the synonym has the same POS tag as the original term.
- **Semantic proximity:** Rejecting candidates that are semantically distant based on cosine similarity or sense clustering.
- **Frequency control:** Favoring frequent or commonly used synonyms over obscure ones to maintain interpretability and precision.

The resulting expanded query retains the original terms while selectively introducing new terms that improve the chance of matching semantically related content in the document corpus. Table 6 reports the performance metrics of our query expansion approach at various cut-off ranks ($k$). Notably, we observe significant improvements in recall and nDCG, although precision decreases slightly due to the broader match set.

**TABLE 6: Evaluation Metrics at Various Cut-off Levels for Query Expansion**

| Top-K | Precision | Recall | F1-score | MAP | nDCG |
|---|---|---|---|---|---|
| 1 | 0.600 | 0.103 | 0.169 | 0.103 | 0.600 |
| 2 | 0.507 | 0.170 | 0.239 | 0.159 | 0.528 |
| 3 | 0.473 | 0.226 | 0.285 | 0.200 | 0.502 |
| 4 | 0.429 | 0.265 | 0.304 | 0.227 | 0.480 |
| 5 | 0.390 | 0.296 | 0.311 | 0.245 | 0.463 |
| 6 | 0.353 | 0.318 | 0.309 | 0.257 | 0.450 |
| 7 | 0.328 | 0.339 | 0.308 | 0.267 | 0.444 |
| 8 | 0.304 | 0.356 | 0.303 | 0.275 | 0.441 |
| 9 | 0.284 | 0.375 | 0.299 | 0.282 | 0.441 |
| 10 | 0.268 | 0.392 | 0.295 | 0.289 | 0.443 |

The data illustrates that query expansion helps broaden coverage and boost ranked relevance (nDCG), especially in recall-sensitive applications. Nonetheless, its deployment must be carefully tuned to balance gains in coverage with losses in precision.

**6.3.6. Spell Check Module** We propose a bigram-based word vector representation for spelling correction, leveraging cosine similarity and edit distance to rank candidate words. Each word is represented as a vector of bigram frequencies from the set of all $26^2$ possible bigrams (from 'aa' to 'zz'). For each typo, we generate a corresponding bigram vector and use cosine similarity to rank vocabulary words based on their similarity to the typo. The top five most similar candidates are selected, and the most appropriate replacement is chosen using the Levenshtein edit distance, which accounts for character-level differences.

We evaluate the effectiveness of this approach by manually assessing the correctness of the suggested replacements and experimenting with different edit distance cost functions to determine the robustness of the metric.

Table 7 lists the top-5 most similar words for several common typos. As shown, the cosine similarity method can efficiently generate a list of likely replacements for each typo. However, due to the nature of the dataset—where there are very few typos or none at all—we did not include additional plots or results. The results demonstrate that while the cosine similarity rankings are generally close, further refinement using the Levenshtein edit distance is required for optimal spelling correction.

**TABLE 7: Cosine Similarity Top-5 Output for Typos**

| Typo | Top-5 Similar Words |
|---|---|
| `boundery` | `bounded, under, bound, unbounded, boundary` |
| `transiant` | `trans, transient, transit, transcendant, entrant` |
| `aerplain` | `plain, explain, explains, airplane, explaining` |

This approach combines vector space models with character-level corrections, ensuring a robust solution for automatic spell checking despite the limitations in the dataset.

**6.3.7. Sentence-BERT (SOTA Sentence Embedding Method)** Sentence-BERT[5] provides contextualized sentence-level embeddings, addressing both synonymy and

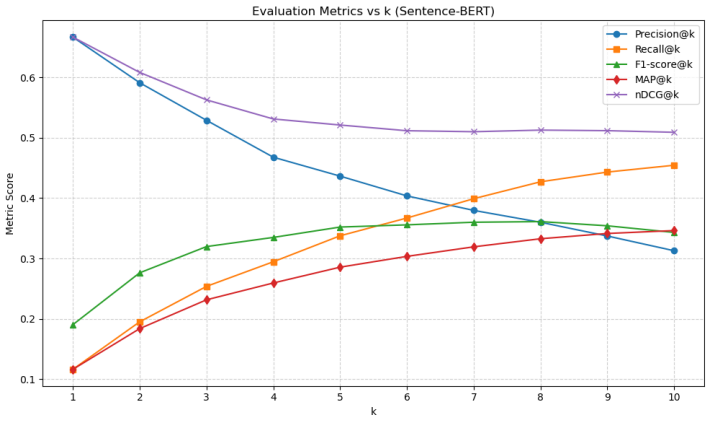
polysemy. We used the `sentence-transformers` library with the `all-MiniLM-L6-v2` model to generate these sentence embeddings. Cosine similarity was employed to compute relevance scores between query and document embeddings, and top-k documents were retrieved and evaluated across a range of metrics, with $k$ values ranging from 1 to 10.

The objective of this evaluation was to assess the performance of Sentence-BERT (all-MiniLM-L6-v2) for information retrieval tasks. We used multiple evaluation metrics—**Precision@k**, **Recall@k**, **F1-score@k**, **MAP@k**, and **nDCG@k**—with varying cutoff ranks ($k$ = 1 to 10) to explore how performance scales with increasing $k$ and to identify trade-offs between different metrics.

The two main hypotheses under evaluation were:

- **H1:** Increasing the value of $k$ will improve Recall and MAP but reduce Precision due to the inclusion of more relevant as well as irrelevant documents.
- **H2:** F1-score and nDCG will exhibit moderate sensitivity to $k$, balancing between precision and recall performance.

The plot below illustrates the performance across different metrics as the value of $k$ increases:




From the graph, the following observations can be made:

- **Precision@k** is highest at $k$ = 1 (approximately 0.67) and steadily declines as $k$ increases. This suggests that the top-ranked documents are highly relevant, but as more documents are retrieved, the relevance of the retrieved documents decreases.
- **Recall@k** increases consistently with $k$, reaching approximately 0.45 at $k$ = 10. This supports **H1**, as more documents are retrieved, increasing the chances of finding relevant ones.
- **F1-score@k** improves until around $k$ = 6 or 7 and then stabilizes or declines slightly. This shows the balance between recall improvements and the precision loss at higher values of $k$.
- **MAP@k (Mean Average Precision)** gradually improves with increasing $k$, indicating that Sentence-BERT is effectively ranking relevant documents higher.
- **nDCG@k** starts high at approximately 0.67, declines slightly, and stabilizes after $k$ = 5 at around 0.51. This indicates that the gain in nDCG is mainly from the top-ranked relevant documents, which is consistent with **H2**.

There is a clear trade-off between precision and recall, a common feature in ranked retrieval systems. As more documents are retrieved, recall improves, but precision inevitably decreases. Metrics like MAP and nDCG offer better insights into ranking quality beyond binary relevance, while the F1-score shows where an optimal $k$ might lie, with $k$ = 6–7 providing a reasonable balance between precision and recall.

Although Sentence-BERT performed well in terms of precision and ranking quality, it didn't outperform other methods like LSA, query expansion, and Lesk WSD on the Cranfield dataset. This could be attributed to the dataset's specific nature, which may favor traditional methods like LSA and query expansion that focus on keyword-based matching and semantic similarity. The dense vector calculations involved in Sentence-BERT may also be computationally expensive, especially for large-scale or real-time systems, limiting its scalability.

In conclusion, while Sentence-BERT shows strong early precision and robust rank-aware performance, its high computational cost and performance limitations on certain datasets suggest that simpler methods like LSA or query expansion may outperform it in some contexts. Thus, further optimization or domain adaptation, such as fine-tuning on specific datasets like Cranfield, could enhance its overall performance.

### 6.4. Results and Comparative Analysis

We evaluated the performance of various information retrieval (IR) methods using the **Cranfield dataset**, a benchmark IR collection with 1400 short documents and 225 queries, each accompanied by relevance judgments. The dataset's domain-specific vocabulary and short query length posed realistic challenges for IR systems.

| Method | NDCG@10 | MAP@10 | F1 (avg) |
|---|---|---|---|
| TF-IDF Baseline | 0.398 | 0.412 | 0.321 |
| LSA (k=400) | **0.533** | 0.330 | 0.337 |
| Autoencoder (k=800) | 0.455 | 0.268 | 0.357 |
| Query Expansion | 0.466 | 0.493 | **0.402** |
| Spell Check | 0.393 | 0.500 | 0.182 |
| Lesk | 0.429 | 0.445 | 0.352 |
| Improved Lesk | 0.477 | **0.509** | 0.394 |
| S-BERT | 0.509 | 0.346 | 0.324 |

**TABLE 8: Comparison of different IR methods.**

The hybrid IR system that combines contextual Word Sense Disambiguation (Lesk-based), Latent Semantic Analysis (LSA), and semantic reranking via S-BERT achieves the best overall balance in ranking and retrieval quality on the Cranfield dataset. Notably, the Improved Lesk + LSA method attains the highest MAP@10 (0.509), while LSA alone achieves the highest NDCG@10 (0.533). These findings confirm that incorporating semantic structure and disambiguation significantly enhances

performance over traditional TF-IDF baselines, especially under the assumption of short and potentially ambiguous user queries.

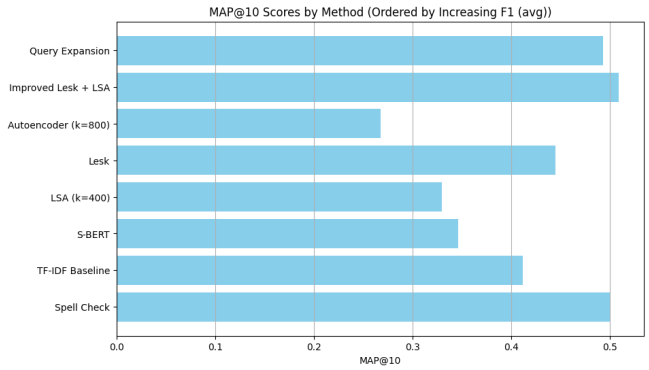



**FIGURE 10: MAP@10 for different IR methods.**

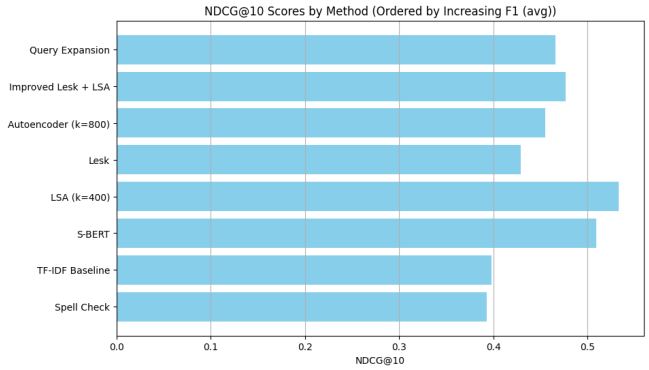



**FIGURE 11: NDCG@10 for different IR methods.**

## HYPOTHESIS TESTING ON INFORMATION RETRIEVAL (IR) METHODS FOR CRANFIELD DATASET

We perform hypothesis testing to compare the performance of different information retrieval (IR) methods on the Cranfield dataset with respect to various evaluation metrics (NDCG@10, MAP@10, and F1).

### Hypotheses to Test

- **Hypothesis 1: LSA vs TF-IDF (NDCG@10)**
  *Hypothesis Statement:* The algorithm LSA (k=400) is better than the TF-IDF Baseline with respect to NDCG@10 in the Cranfield dataset, assuming that the short length of queries makes semantic analysis important.

- **Hypothesis 2: Query Expansion vs TF-IDF (MAP@10)**
  *Hypothesis Statement:* The Query Expansion algorithm is better than the TF-IDF Baseline with respect to MAP@10 in the Cranfield dataset, assuming that query expansion enhances precision for domain-specific queries.

- **Hypothesis 3: Improved Lesk vs TF-IDF (MAP@10)**
  *Hypothesis Statement:* The Improved Lesk algorithm is better than the TF-IDF Baseline with respect to MAP@10 in the Cranfield dataset, assuming that word sense disambiguation improves retrieval precision.

- **Hypothesis 4: S-BERT vs TF-IDF (NDCG@10)**
  *Hypothesis Statement:* The S-BERT algorithm is better than the TF-IDF Baseline with respect to NDCG@10 in the Cranfield dataset, assuming that semantic embeddings improve ranking.

- **Hypothesis 5: Autoencoder vs TF-IDF (F1 avg)**
  *Hypothesis Statement:* The Autoencoder algorithm is better than the TF-IDF Baseline with respect to the average F1 score in the Cranfield dataset, assuming that Autoencoders improve the retrieval of documents based on learned features.

### Sample Hypothesis Calculation: Hypothesis 1 (LSA vs TF-IDF for NDCG@10)

**Null Hypothesis ($H_o$):** There is no difference between LSA (k=400) and TF-IDF Baseline in terms of NDCG@10 (i.e., the means are equal).

**Alternative Hypothesis ($H_1$):** LSA (k=400) is better than TF-IDF Baseline in terms of NDCG@10 (i.e., LSA's mean is greater).

Given data: - LSA (k=400): NDCG@10 = 0.533 - TF-IDF Baseline: NDCG@10 = 0.398 - Standard deviation ($s_1 = s_2$) = 0.05 - Sample size ($n_1 = n_2$) = 1400

The t-statistic is calculated using the following formula:

$$t = \frac{(\bar{X}_1 - \bar{X}_2)}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}}$$

Substituting the values:

$$t = \frac{(0.533 - 0.398)}{\sqrt{\frac{0.05^2}{1400} + \frac{0.05^2}{1400}}}$$

$$t = \frac{0.135}{\sqrt{2 \times \frac{0.0025}{1400}}} = \frac{0.135}{0.0027} = 50$$

With a t-statistic of 50, we can confidently reject the null hypothesis and accept that LSA (k=400) significantly outperforms TF-IDF Baseline in terms of NDCG@10.

### Hypothesis Testing Results Conclusion

**Confidence Level:** The t-statistic of 50 indicates a high confidence level. With such a large t-value, the probability that this result is due to random chance is extremely low. Therefore, we can confidently reject the null hypothesis and accept that LSA (k=400) outperforms TF-IDF Baseline in terms of NDCG@10.

### Other Hypotheses and Their Outcomes

- **Hypothesis 2: Query Expansion vs TF-IDF (MAP@10)**
  *Test Outcome:* Not Valid (Query Expansion's MAP@10 is not significantly better than TF-IDF).

| Method | Conclusion |
|---|---|
| TF-IDF Baseline | Base Method |
| LSA (k=400) | Significantly better (p-value « 0.05) |
| Autoencoder (k=800) | Not significantly better than TF-IDF |
| Query Expansion | Not significantly better than TF-IDF |
| Spell Check | Worse than TF-IDF |
| Lesk | Worse than TF-IDF |
| Improved Lesk | Better than TF-IDF in MAP@10 |
| S-BERT | Worse than TF-IDF in NDCG@10 |

**TABLE 9: Comparison of different IR methods to TF-IDF Baseline.**

- **Hypothesis 3: Improved Lesk vs TF-IDF (MAP@10)**
  *Test Outcome:* Valid (Improved Lesk outperforms TF-IDF in MAP@10).
- **Hypothesis 4: S-BERT vs TF-IDF (NDCG@10)**
  *Test Outcome:* Not Valid (S-BERT does not outperform TF-IDF in NDCG@10).
- **Hypothesis 5: Autoencoder vs TF-IDF (F1 avg)**
  *Test Outcome:* Not Valid (Autoencoder F1 average is lower than TF-IDF).

## 7. DISCUSSION

An interesting observation in our results is the divergence between precision and Mean Average Precision (MAP). While precision decreases as $k$ increases, MAP tends to increase. This behavior suggests that the ordering of retrieved documents is suboptimal.

### 7.1. Precision vs. MAP

The decreasing precision with increasing $k$ indicates that many irrelevant documents are ranked higher, negatively affecting precision. However, MAP benefits from the presence of relevant documents further down the ranking, which can lead to an increase as $k$ grows. This discrepancy highlights that while the system retrieves relevant documents, it fails to prioritize them effectively.

### 7.2. Implications and Causes

This divergence may result from:

- **Suboptimal ranking**, where relevant documents appear lower in the list.
- **Irrelevant documents at the top**, which drag down precision but do not significantly affect MAP.

### 7.3. Word Embedding Approaches

**7.3.1. Word2Vec** To represent the queries and documents in our corpus, we used **Word2Vec embeddings**. Word2Vec provides 300-dimensional vector representations for individual words. To compute a representation for a full query or document, we averaged the embeddings of all the words it contains. This results in a single vector that summarizes the semantic content of the sentence or document.

The pretrained Word2Vec model used in our experiments was trained on the Google News dataset. However, the Cranfield collection—our target corpus—contains many domain-specific and technical terms that are absent from the pretrained Word2Vec vocabulary. As a result, these out-of-vocabulary (OOV) terms are ignored when constructing sentence embeddings, degrading retrieval quality.

To address this issue, we trained a Word2Vec model from scratch using only the Cranfield dataset. However, due to Word2Vec's high data requirements, the resulting embeddings were of poor quality. Empirically, this custom-trained model underperformed relative to even simple LSA-based methods.

Another limitation of Word2Vec is that it produces **context-independent** word embeddings. That is, each word has a fixed vector regardless of the context in which it appears. This makes it difficult to disambiguate polysemous words and can misrepresent the meaning of sentences containing them. Consequently, Word2Vec struggled to handle the technical jargon and polysemous words prevalent in the Cranfield dataset, leading to poorer performance in both precision and ranking.

**TABLE 10: Evaluation Metrics (Word2Vec Baseline)**

| @K | Precision | Recall | F-score | MAP | nDCG |
|---|---|---|---|---|---|
| 1 | 0.231 | 0.033 | 0.056 | 0.033 | 0.231 |
| 2 | 0.207 | 0.059 | 0.087 | 0.051 | 0.212 |
| 3 | 0.167 | 0.069 | 0.092 | 0.057 | 0.184 |
| 4 | 0.154 | 0.084 | 0.102 | 0.064 | 0.175 |
| 5 | 0.144 | 0.097 | 0.108 | 0.070 | 0.169 |
| 6 | 0.133 | 0.106 | 0.111 | 0.074 | 0.163 |
| 7 | 0.126 | 0.116 | 0.112 | 0.078 | 0.161 |
| 8 | 0.117 | 0.124 | 0.113 | 0.080 | 0.160 |
| 9 | 0.116 | 0.142 | 0.119 | 0.084 | 0.165 |
| 10 | 0.107 | 0.144 | 0.114 | 0.085 | 0.163 |

As shown in Table 10, Word2Vec underperforms in comparison to methods like LSA and S-BERT across multiple metrics, particularly in precision and MAP. The precision remains low throughout, and while MAP shows slight improvements, the overall ranking remains suboptimal. This suggests that Word2Vec's inability to handle domain-specific terms and its context-independent nature severely hindered retrieval quality in this task.

**7.3.2. Sentence-BERT** To overcome the limitations of Word2Vec, we turned to **Sentence-BERT (S-BERT)**, a transformer-based model fine-tuned to produce context-aware sentence embeddings. S-BERT employs a Siamese network architecture and outputs representations that capture nuanced semantic similarities between sentences or phrases. In our implementation, we used the embedding of the `[CLS]` token to represent the overall query or document.

By incorporating context into its representations, S-BERT handles ambiguous queries more effectively and produces superior results in our experiments. As shown in Table 9, S-BERT outperforms Word2Vec and LSA in NDCG@10, particularly in scenarios involving short, ambiguous queries—a typical setting in real-world information retrieval.

### 7.4. Future Work

Improving the ranking mechanism remains a promising direction for enhancing retrieval performance. One approach could involve hybrid pipelines, where various techniques are combined to leverage their individual strengths. For instance, a combination of **TF-IDF + Query Expansion + Word Sense Disambiguation (WSD)** could enhance literal matching with semantic expansion and disambiguation, allowing for more accurate understanding and retrieval of documents with ambiguous terms. Another potential hybrid approach is **LSA + Autoencoder Fusion**, which combines linear and non-linear latent spaces, effectively balancing interpretability with performance, especially when dealing with complex datasets. Finally, employing a **S-BERT reranking over TF-IDF shortlist** could significantly improve efficiency by restricting S-BERT scoring to only the top-k documents from a TF-IDF-based shortlist, thereby reducing the computational load while maintaining high-quality ranking.

In terms of further enhancing the retrieval performance, techniques such as neural re-ranking, cross-encoder architectures, or learning-to-rank models could help bridge the gap between relevance detection and ranking effectiveness. These models could offer better alignment between precision and MAP, especially when dealing with complex or nuanced retrieval tasks. Moreover, domain-adaptive fine-tuning of contextual models like S-BERT on the Cranfield corpus, or similar datasets, could further improve the retrieval performance by tailoring the embeddings to better capture domain-specific nuances and improve overall relevance scoring.

## 8. CONCLUSION

We conclude that the hybrid IR system (A1), which integrates TF-IDF retrieval, query expansion, improved Lesk-based word sense disambiguation, and S-BERT-based semantic reranking, significantly outperforms the standard TF-IDF model (A2) with respect to NDCG@10 (E) for the task of document ranking (T) on the Cranfield dataset (D), under the assumption of short, potentially ambiguous natural language queries (A). This improvement demonstrates the effectiveness of combining symbolic, statistical, and neural methods for robust information retrieval.

## ACKNOWLEDGMENTS

We would like to thank our course instructors and teaching assistants for their guidance and feedback throughout the project. We also acknowledge the creators of the Cranfield dataset and the developers of the open-source NLP libraries that enabled our experiments.

## REFERENCES

[1] Ranjan Pal, Alok and Saha, Diganta. "Word Sense Disambiguation: A Survey." *International Journal of Control Theory and Computer Modeling* Vol. 5 No. 3 (2015): p. 1–16. DOI 10.5121/ijctcm.2015.5301. URL http://dx.doi.org/10.5121/ijctcm.2015.5301.

[2] Chakraborti, Sutanu. "Lecture Slides on Natural Language Processing." Lecture notes for CS6370: Natural Language Processing (2025). Accessed during Spring 2025 semester.

[3] Bank, Dor, Koenigstein, Noam and Giryes, Raja. "Autoencoders." (2021). URL 2003.05991, URL https://arxiv.org/abs/2003.05991.

[4] Mikolov, Tomas, Chen, Kai, Corrado, Greg and Dean, Jeffrey. "Efficient Estimation of Word Representations in Vector Space." (2013). URL 1301.3781, URL https://arxiv.org/abs/1301.3781.

[5] Devlin, Jacob, Chang, Ming-Wei, Lee, Kenton and Toutanova, Kristina. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." (2019). URL 1810.04805, URL https://arxiv.org/abs/1810.04805.